

Multi-Tenant Special Therapist Platform

High-level architecture, onboarding flow, data model, and PostgreSQL starter design

Purpose

This document packages the initial product blueprint for a SaaS platform that allows multiple therapy companies to register, create branches or service locations, manage therapists and schedules, accept customer bookings, process payments, and operate services in a tenant-isolated environment.

Prepared for	Product planning / system design
Business model	Multi-tenant service platform
Target output	PDF concept document

1. Executive Summary

The proposed application is a multi-tenant digital platform for special therapist businesses. Each company registers as a tenant, manages one or more branches or service areas, defines therapist rosters and schedules, publishes services, accepts customer bookings, and runs day-to-day operations through a shared platform with isolated data.

- Platform owner controls tenant onboarding, subscription, policies, and operational visibility.
- Tenant admin controls branch setup, therapist profiles, services, availability, staff users, and reporting.
- Customers browse available locations or service types, select a therapist or time slot, make bookings, pay, and receive notifications.
- Operations staff manage assignment, rescheduling, completion, invoicing, and service follow-up.

2. Primary Actors and Scope

Actor	Role in Platform	Main Responsibilities
Platform Admin	Owns the SaaS	Approve tenants, manage packages, monitor usage, govern policy
Tenant Admin	Owns one company	Configure branches, services, therapist roster, pricing, and staff users
Ops Staff	Runs service delivery	Confirm appointments, assign therapist, manage changes, close sessions
Customer	Books service	Browse, book, pay, receive reminders, view history

3. High-Level Architecture

The platform separates channel access, business services, persistence, and third-party integrations. This keeps tenant onboarding, booking, and operational workflows modular without requiring a fully distributed microservice implementation on day one.

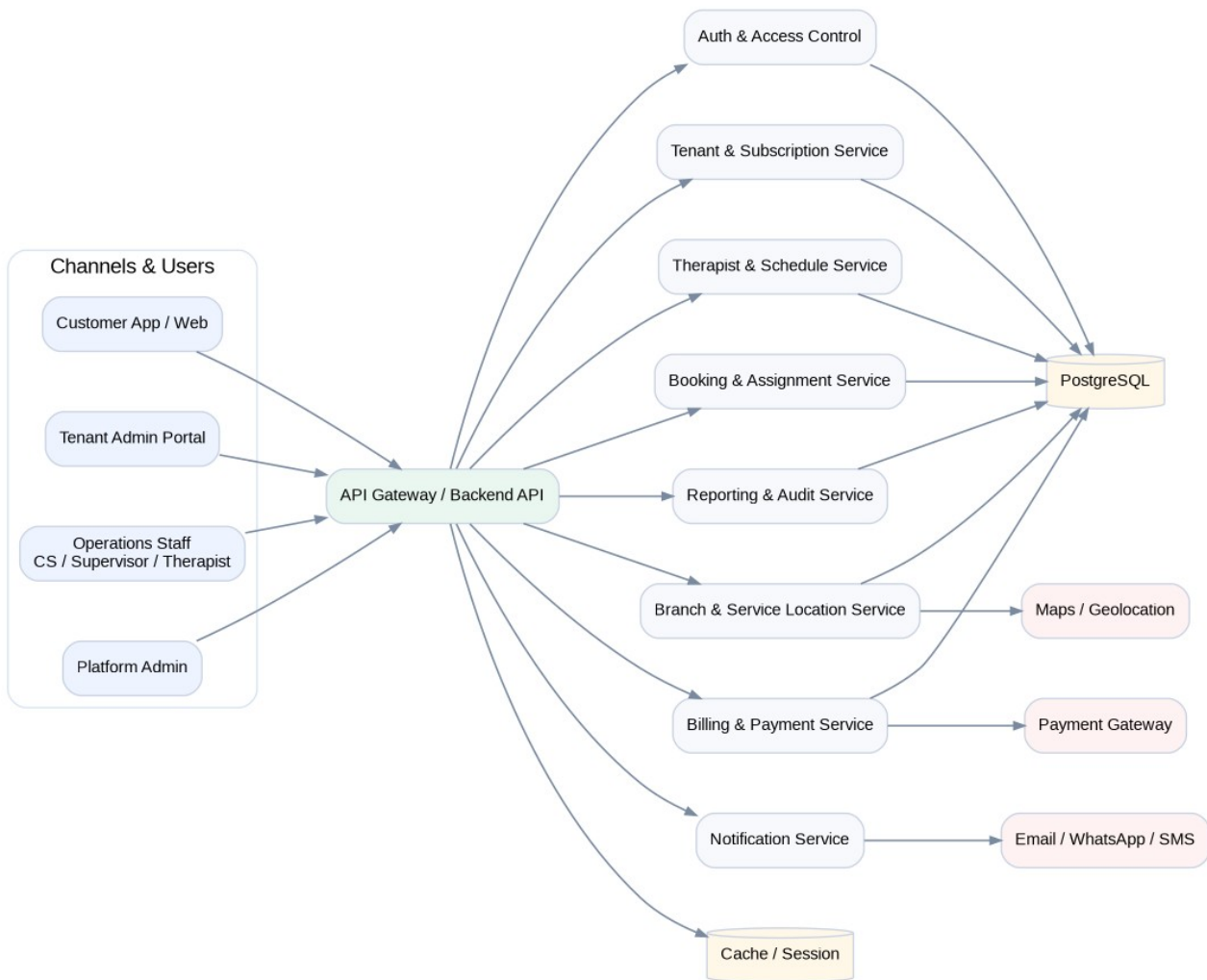


Figure 1. High-level architecture for the multi-tenant special therapist platform.

4. End-to-End Business Flow

The commercial and operational life cycle starts with company registration and ends with service completion, payment closure, and reporting. The core value chain is shown below.

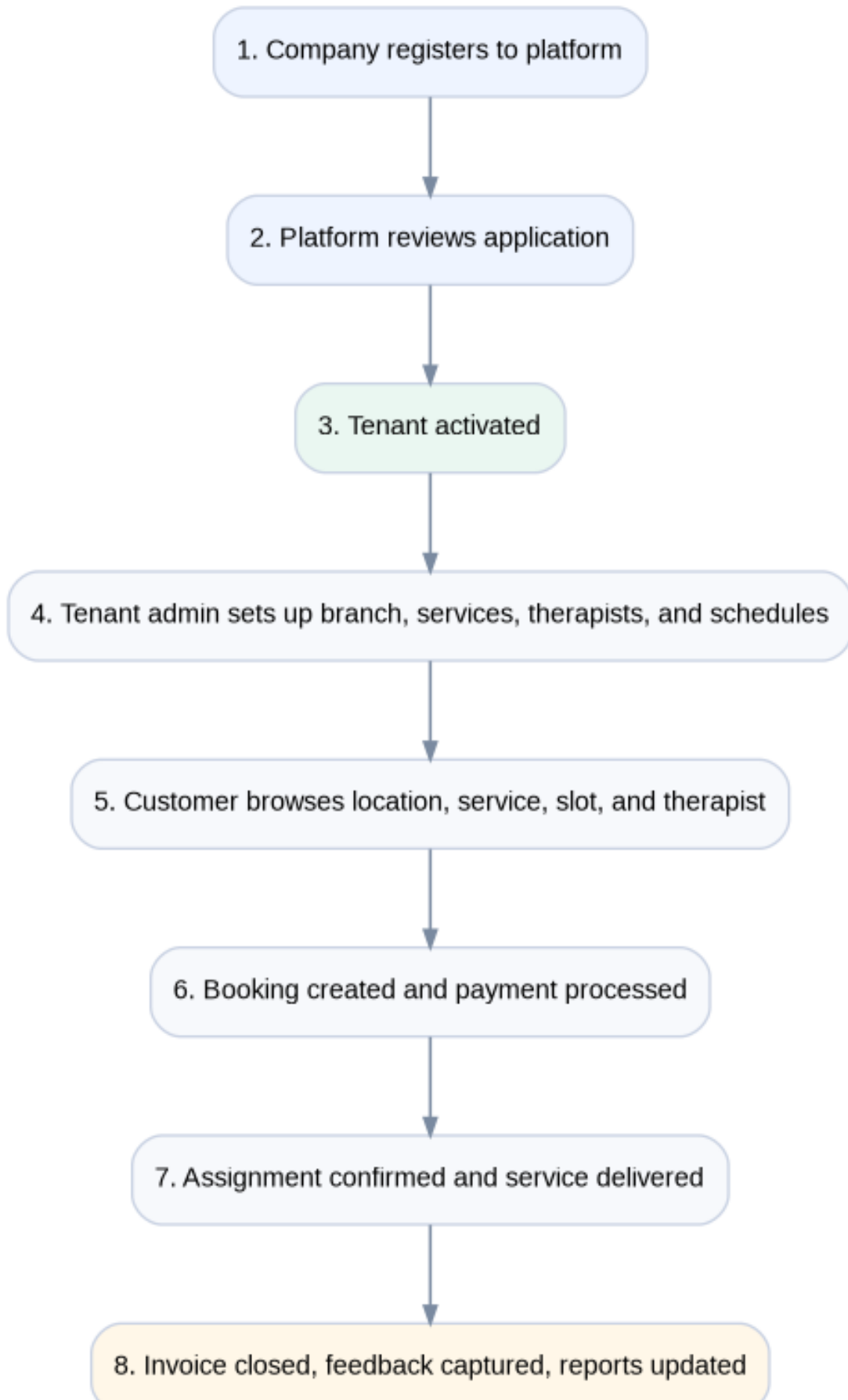


Figure 2. Business flow from tenant registration to completed service.

5. Tenant and Branch Onboarding Flow

A branch or service location should only become public after tenant approval, package validation, branch setup, service setup, therapist setup, and schedule configuration are complete.

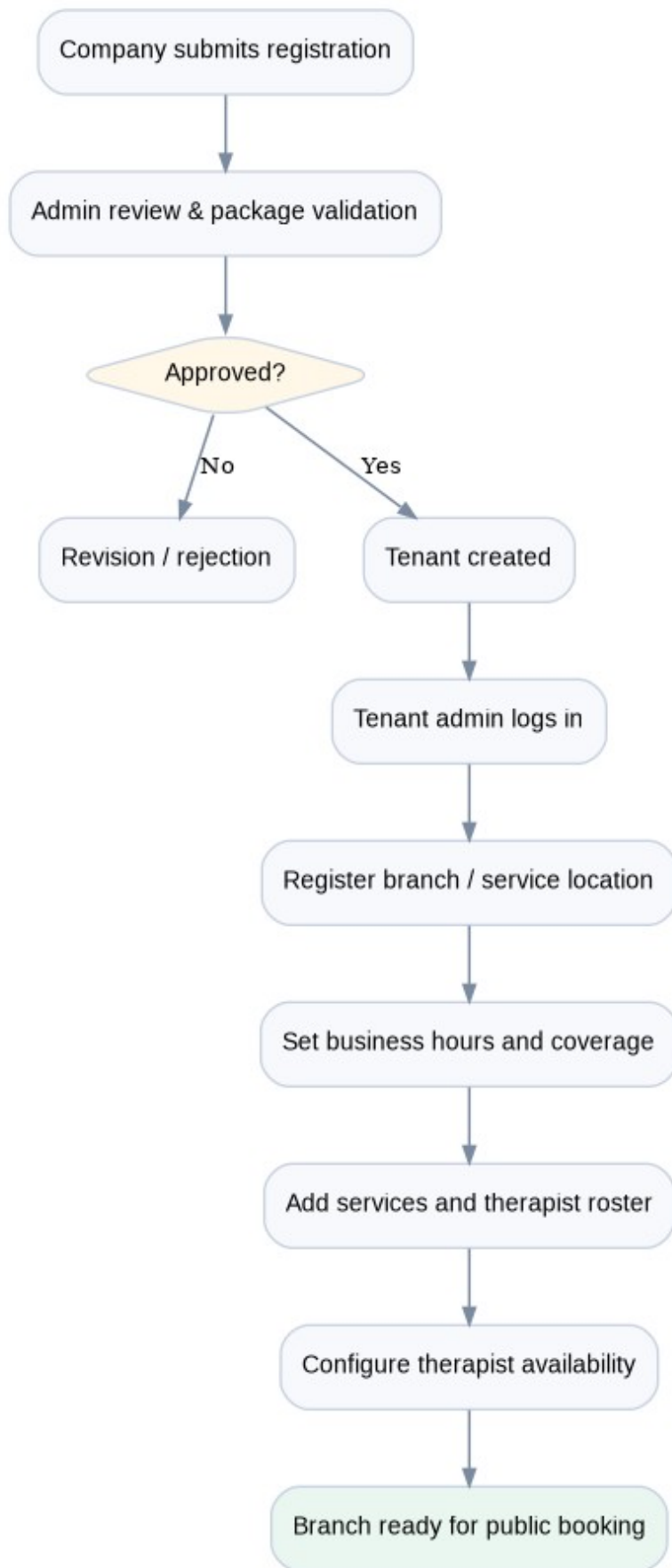


Figure 3. Tenant registration and branch onboarding process.

6. Core Modules

Module	Scope	Priority	Notes
Tenant Management	Registration, approval, subscription, company profile	MVP	Mandatory for SaaS onboarding
Branch Management	Locations, business hours, coverage, status	MVP	Supports in-branch and service-area operations
Service Catalog	Service definitions, pricing, duration, category	MVP	Drives booking selection and billing
Therapist Management	Therapist profile, skills, active status	MVP	Base roster per tenant and branch
Availability & Scheduling	Work schedule, blocked slots, leave, allocation	MVP	Required to prevent double booking
Booking & Assignment	Customer booking, confirmation, assignment, status	MVP	Central operational module
Billing & Payment	Invoice, online payment, payment tracking	MVP	Supports deposit or full payment
Notifications	Email / WhatsApp / SMS reminders	Phase 2	Can start with a minimal provider integration
Reporting & Audit	Revenue, occupancy, therapist utilization, logs	Phase 2	Add dashboards after core flows stabilize

7. Conceptual Data Model

The data model centers on tenant isolation. All operational records should carry tenant context, while branch context is attached where location-specific activity must be tracked.

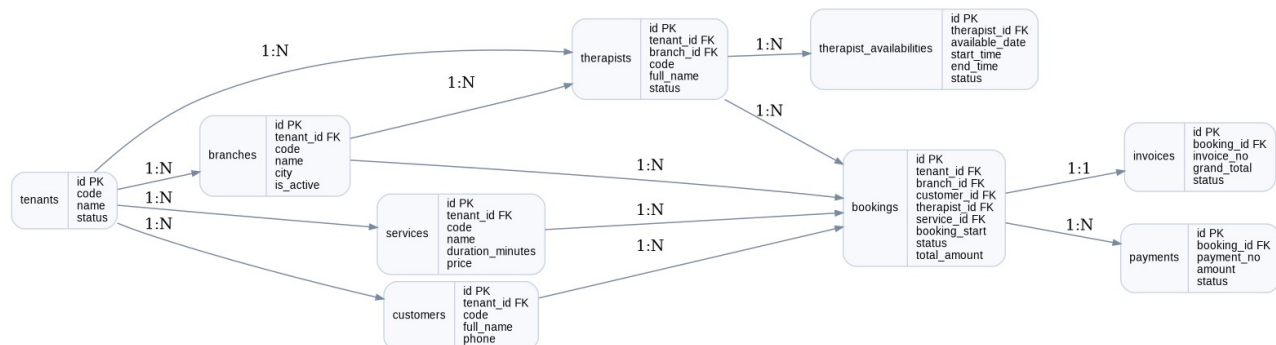


Figure 4. Conceptual ERD for the starter relational model.

8. PostgreSQL Starter Table Catalog

The following table set is sufficient for an MVP that supports tenant onboarding, branch setup, therapist scheduling, customer booking, invoicing, and payment recording.

Table	Priority	Purpose
tenants	MVP	Stores company-level tenant identity and lifecycle status
tenant_subscriptions	MVP	Tracks plan, validity period, and usage limits
branches	MVP	Stores branch or service-location information
roles	MVP	Role master for platform and tenant users
users	MVP	Tenant or platform users with branch scope when relevant
services	MVP	Service catalog with price and duration
therapists	MVP	Therapist roster and employment status
therapist_services	MVP	Maps therapist capability to service catalog
therapist_availabilities	MVP	Daily slot availability, blocked time, leave
customers	MVP	Customer master data and contact channels
bookings	MVP	Primary appointment transaction and lifecycle status
booking_items	Phase 2	Optional breakdown for add-ons or multiple service lines
service_requests	Phase 2	Captures extra requests, rescheduling notes, or special handling
invoices	MVP	Billing summary for a booking
payments	MVP	Payment attempts and settlement records
notifications	Phase 2	Delivery log for transactional messages
audit_logs	Phase 2	Operational and security trail

9. Key Business Rules and Design Decisions

- Every operational table should carry `tenant_id` to guarantee tenant isolation and simplify authorization filters.
- Branch visibility should depend on tenant approval, subscription validity, and branch activation status.
- Therapist availability must be modeled explicitly to prevent overlapping bookings and to support leave or blocked time.
- Booking status should at minimum support draft, `waiting_payment`, confirmed, assigned, `in_progress`, completed, cancelled, and `no_show`.
- Invoices should be one-to-one with a booking in the starter design, while payments may be one-to-many to support deposits or split settlement.
- Customer access should be limited to their own booking history, profile, invoices, and notifications.

10. Security, Privacy, and Operational Controls

Recommended baseline controls

- Role-based access control for platform admin, tenant admin, branch admin, operations staff, therapist, and customer roles.
- Strict row-level filtering by tenant and branch scope inside APIs and background jobs.
- Audit logging for booking changes, therapist assignment, payment updates, and status overrides.
- Protection of personally identifiable information, especially customer address, phone, medical preference notes, and therapist assignment history.
- Notification templates that avoid exposing sensitive information in unsecured channels.

11. MVP Delivery Scope

Phase	Outcome	Included Capabilities
MVP	Core booking platform	Tenant approval, branches, services, therapists, availability, booking, invoice, payment log
Phase 2	Operational depth	Notifications, richer reporting, add-on requests, audit logs, promotions
Phase 3	Service optimization	Advanced routing, loyalty, therapist utilization dashboards, CRM integrations

12. Assumptions for the Initial Implementation

This blueprint assumes a shared-application multi-tenant architecture, with tenant separation enforced logically through `tenant_id` and authorization rules. It also assumes a web-first admin portal, a customer-facing mobile or responsive web channel, and a PostgreSQL-backed transactional system.

- One company equals one tenant account within the platform.
- A tenant may operate one or more branches or service locations.
- Service can be delivered at branch or at a customer-defined location if the business model allows it.

- Therapist assignment can be automatic or manual depending on policy.
- Payment gateway integration may initially be asynchronous, with status polling or webhook confirmation.

13. Recommended Next Documents

- Detailed ERD with full attributes, indexes, unique constraints, and foreign keys.
- API contract list for admin, operations, and customer channels.
- Role-permission matrix per module and per action.
- Booking lifecycle and payment reconciliation specification.
- UI wireframes for tenant onboarding, branch setup, and booking journeys.